

Drake AI Enterprise Platform

Module-wise technical stack, model usage, and UI technology report generated from the repository code.

Executive Summary

- Architecture: React/Vite frontend, FastAPI backend, SQLAlchemy data layer, Docker Compose infrastructure.
- Primary AI copilot model: Anthropic Claude Sonnet 4, model id claude-sonnet-4-20250514.
- Predictive results are mainly produced by scikit-learn, optional XGBoost, TensorFlow/Keras, PyWavelets, FFT, and deterministic petrophysical formulas.
- The UI stack is React 18, TypeScript, Vite, Tailwind CSS, Plotly, Recharts, TanStack Query, Zustand, Axios, React Router, and lucide-react.

Layer	Technology	Where configured	Notes
Frontend	React 18 + Vite + TypeScript	frontend/package.json, frontend/Dockerfile	Port 3000 in Docker/dev scripts.
Backend	FastAPI Python 3.11 style stack	backend/requirements.txt, backend/app/main.py	Port 8000; Swagger at /docs; routers under /api/.*
Database	SQLite by default, PostgreSQL 15 in Docker	backend/app/core/config.py, docker-compose.yml	DATABASE_URL defaults to sqlite:///./drakeai.db; Docker uses postgres service.
Cache/session	Redis 7 optional	docker-compose.yml, petrophysics RedisBackedSessionStore	Used for petrophysics sessions when available, local cache fallback otherwise.
Object/file storage	MinIO plus local uploads	docker-compose.yml, backend settings	MinIO config exists; many modules also write to uploads/.
Migrations	Alembic	backend/alembic.ini, backend/migrations	Docker command runs alembic upgrade head.
Containerization	Docker Compose	docker-compose.yml, docker-compose.prod.yml	Services: postgres, redis, minio, backend, frontend.

Model Usage For Results

This section lists model names and algorithms explicitly found in the inspected code paths.

Result area	Model actually referenced in code	Used for
Drake GPT	Anthropic claude-sonnet-4-20250514	Context-aware petrophysics assistant chat when ANTHROPIC_API_KEY is configured.
Drake GPT fallback	Drake GPT Local	Rule-based responses without external API key.
Missing log prediction	Extra Trees, Random Forest, Gradient Boosting	Filling missing LAS log intervals and returning R2/RMSE/MAE-style metrics in UI.
AI parameter prediction	Random Forest AI, XGBoost AI, Gradient Boosting AI, Decision Tree AI, empirical equations	VSH, porosity, saturation, permeability, lithology and uncertainty rows.
AI uncertainty	Random Forest tree spread; XGBoost/boosting bootstrap ensembles; manual percent fallback	P10/P50/P90 uncertainty bands and confidence/reliability summaries.
Seismic enhancement	Conv1D U-Net; PyWavelets + RandomForest; Conv1D frequency-band autoencoder; FFT fallback	2D/3D seismic frequency enhancement, amplitude panels, spectra, and SEG-Y export where possible.
Legacy AI jobs	LSTM + Random Forest Ensemble; Random Forest Classifier; CNN Gradient-Based Boundary Detector; Gradient Boosting + Core Calibration; Random Forest + FZI; Archie + Neural Network Sw	Job result labels returned by backend/app/ml/petrophysics.py for demo/enterprise job workflows.
CCUS	Rule-based screening and quality scoring	CO2 storage candidates, seal/caprock candidates, reservoir-seal pairs, and XLSX export.
Geothermal and production	Analytical calculations/KPI rules in inspected route layer	Screening summaries, production analysis, exports and visualizations.

Separate Module Stack Usage

Module	Code location	Technical stack / libraries	Purpose and result model
Authentication and security	backend/app/api/auth.py, backend/app/core/security.py	FastAPI, SQLAlchemy, Pydantic, python-jose JWT, passlib bcrypt	Login and token-secured API access. No ML model; deterministic auth logic.
Projects, wells, curves, files	backend/app/api/projects.py, wells.py, curves.py, files.py; backend/app/services	FastAPI routers, SQLAlchemy ORM, lasio, pandas, local uploads, optional MinIO	Core data-management layer for LAS curves and project assets. No predictive model; stores measured and predicted curves.
AI jobs	backend/app/api/ai_jobs.py, backend/app/ml/petrophysics.py	FastAPI job API, SQLAlchemy job records, NumPy, pandas, scikit-learn imports	Runs named analysis jobs. Reported result models include LSTM + Random Forest Ensemble, Random Forest Classifier, CNN Gradient-Based Boundary Detector, Gradient Boosting + Core Calibration, Random Forest + FZI Calibration, Archie + Neural Network Sw Model, and Depth-Match + Overlap Merge.
Drake GPT copilot	backend/app/api/gpt.py	FastAPI, Anthropic Python SDK, SQLAlchemy well context	Primary LLM: claude-sonnet-4-20250514. Fallback model label: Drake GPT Local rule-based response engine.
Petrophysics LAS analysis	backend/app/api/petrophysics.py	lasio, pandas, NumPy, SciPy stats and KDE, Plotly-compatible JSON, Redis-backed session cache	Crossplots, histograms, LAS parsing, statistics, quality labels, and session-based workflows. Mostly analytical/statistical, not a trained persistent model.
Missing log prediction	backend/app/api/petrophysics.py, frontend/src/pages/UIOnlyModulePage.tsx	scikit-learn ExtraTreesRegressor, RandomForestRegressor, GradientBoostingRegressor; r2_score; pandas/NumPy	Predicts missing curve intervals from uploaded LAS features. UI exposes Extra Trees, Random Forest, and Gradient Boosting; output includes R2, predicted samples, CSV, LAS, and Plotly preview.
AI parameter prediction	backend/app/ml/drake_prediction_standalone.py, Drake_Uncertainty/app.py	NumPy, pandas, scikit-learn RandomForestRegressor, GradientBoostingRegressor, DecisionTreeRegressor, optional XGBoost	Computes VSH, PHIT/PHIE, water saturation, permeability, lithology, confidence, and P10/P50/P90 uncertainty bands. Model choices: empirical, Random Forest AI, XGBoost AI, Gradient Boosting AI, Decision Tree AI.
AI uncertainty	backend/app/ml/drake_uncertainty.py, backend/app/ml/drake_uncertainty_standalone.py, Drake_Uncertainty/app.py	NumPy, pandas, SciPy, scikit-learn, optional XGBoost, Plotly output	Produces uncertainty envelopes and P10/P50/P90 curves. Random Forest uses tree distribution; XGBoost and boosting variants use bootstrap/ensemble spread where implemented.
Facies classification and formation tops	backend/app/ml/mini_petrophysics_toolbox.py, frontend specific pages	pandas, NumPy, scikit-learn clustering/classification patterns, Plotly-compatible responses	Facies classification and top detection workflows. Legacy job labels include Random Forest Classifier and CNN Gradient-Based Boundary Detector.
Seismic frequency enhancer	backend/app/api/seismic.py, backend/app/ml/seismic_enhancer.py	NumPy FFT, SciPy ndimage, PyWavelets, scikit-learn RandomForestRegressor/MinMaxScaler, TensorFlow/Keras Conv1D, segyio	Models used: 2D TensorFlow Conv1D U-Net, 3D PyWavelets + RandomForest low-frequency ML, 3D TensorFlow Conv1D frequency-band autoencoder, combined RF + Conv1D DL, or FFT band enhancement fallback.
Geothermal screening	backend/app/api/geothermal.py, backend/app/ml/geothermal.py	FastAPI, pandas, LAS parsing utilities, CSV/JSON export, generated PNG heat-flow map	Log-based geothermal interpretation and heat-flow outputs. Primarily analytical rules/calculations; no explicit named ML model found in the inspected route.
Production intelligence	backend/app/api/production.py, backend/app/ml/production_intelligence.py	FastAPI, pandas, CSV/XLSX ingestion, analytical production KPIs	Production sample/analyze endpoint. Result logic is production analytics rather than an explicit trained model in the route.

Module	Code location	Technical stack / libraries	Purpose and result model
CCUS preliminary screening	backend/ccus/router.py, backend/ccus/core.py	FastAPI, custom LAS parser, rule-based petrophysical calculations, Plotly-compatible log tracks, XLSX export writer	Screening model is deterministic cutoff/rule-based reservoir-seal evaluation using GR, RHOB, NPHI, RT, PHIE, VSH, permeability proxies, and quality scoring.
Reports and exports	backend/app/api/reports.py, app services/export_manager.py, Drake_Uncertainty/utlils/repor t_utils.py	FastAPI, fpdf2, openpyxl, Pillow, pandas, file responses	Generates PDF, spreadsheet, CSV/JSON, and report exports. No ML model; packages model outputs for users.
Standalone Drake_Uncertainty app	Drake_Uncertainty/app.py, templates, static	Flask, Jinja templates, Plotly, lasio, pandas, NumPy, SciPy, scikit-learn, optional XGBoost	Standalone dashboard for LAS upload, prediction, uncertainty, exports, and visualizations. Models: Random Forest, XGBoost, Gradient Boosting, Decision Tree, heuristic empirical calculations.

Complete UI Technical Stack

UI area	Frontend files	UI technical stack	Backend/API linkage
Application shell and routing	frontend/src/App.tsx, components/layout	React 18, TypeScript, Vite 5, React Router DOM, TanStack Query, Zustand, React Hot Toast, lucide-react icons	Routes protected by token state; API calls centralized through frontend/src/services/api.ts.
Styling and build	frontend/src/index.css, tailwind.config.js, postcss.config.js, vite.config.ts	Tailwind CSS 3, PostCSS, Autoprefixer, CSS modules/inline styles, Vite dev server and production build	Docker frontend exposes port 3000 and uses VITE_API_URL for backend URL.
Visualization components	components/petrophysics, UIOnlyModulePage.tsx	plotly.js-dist-min, Recharts, custom tables, upload panels, histogram/crossplot viewers	Consumes /api/petrophysics, /api/seismic, /api/production, /api/ccus, /api/geothermal.
Data and files	services/api.ts, utils/localProjectStorage.ts, drakeProjectFile.ts	Axios, browser local/session storage, React Dropzone, typed API wrappers	Uploads LAS, SEG-Y/NPY/CSV/TXT, production CSV/XLSX, and saves project/module results.
Main user modules	pages/*.tsx	Dashboard, projects, data management, reports, settings, petrophysics modules, seismic enhancer, production intelligence, CCUS, geothermal, digitizer placeholders	Most heavy computation lives in FastAPI; frontend handles workflow state, controls, tables, and plots.

Key UI Routes

- /dashboard, /projects, /data-management, /analytics/reports, /settings
- /petrophysics/log-visualization, /missing-log-prediction, /ai-facies-classification, /ai-formation-tops, /ai-parameter-prediction, /ai-uncertainty, /auto-splicer, /crossplot, /histogram
- /seismic/frequency-enhancer, /production/intelligence, /ccus/ai-preliminary-screening, /geothermal/log-based-screening
- /digitizer/drake-slm-gpt and /digitizer/drake-ocr

Backend API Surface

API prefix	Router/module	Stack and responsibility
/api/auth	auth.py	JWT login and user authentication.
/api/projects	projects.py	Project CRUD and active project workflows.
/api/wells	wells.py	Well records and metadata.
/api/curves	curves.py	Curve listing and curve data retrieval.
/api/files	files.py	LAS/file upload, parsing, and storage integration.
/api/ai	ai_jobs.py	AI job creation, status, model name, result metadata.
/api/reports	reports.py	Report/export endpoints.
/api/gpt	gpt.py	Claude Sonnet 4 copilot and local rule fallback.
/api/petrophysics	petrophysics.py	LAS sessions, crossplots, histograms, missing logs, predictions, uncertainty.
/api/seismic	seismic.py	Seismic upload, inspect, and frequency enhancement.
/api/geothermal	geothermal.py	Geothermal LAS screening and exports.
/api/production	production.py	Production intelligence sample/analyze workflows.
/api/ccus	ccus/router.py	Preliminary CCUS reservoir/seal screening and XLSX export.

Notes And Assumptions

- The report is based on static repository inspection, not live execution of every endpoint.
- Some modules have UI placeholders or demo-style model labels; those are identified as labels when the underlying code returns synthetic/demo predictions.
- Default local database is SQLite, while Docker uses PostgreSQL; both are included because both are present in the code/configuration.
- Misspelling in folder name Drake_Uncertainty is kept as it appears in the repository.